

Agent 自我改进的六条路

原创 John AGI Hunt 2026年4月6日 13:09 北京



AGI Hunt

关注AGI 的沿途风景！ 前网易资深技术专家；出海某垂类 Agent 方向创业中；佛系写作 1815篇原创内容



公众号

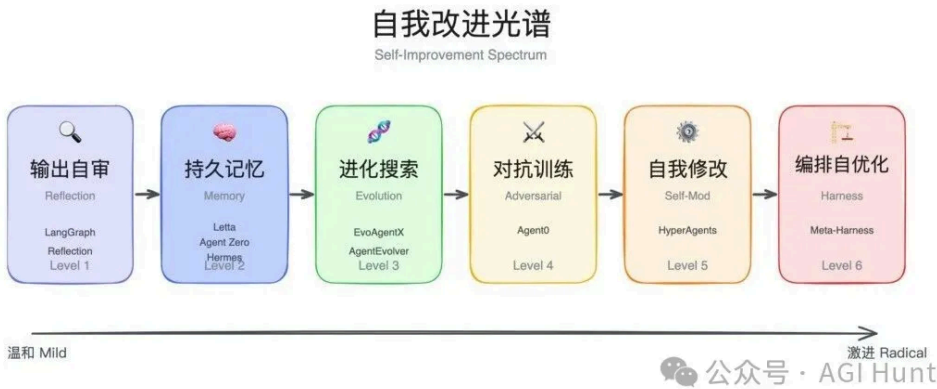
当一个 AI Agent 完成任务之后，它能从这次经历中学到什么吗？

下次遇到类似问题，它能做得更好吗？

这个问题，正在被越来越多的开源项目认真回答。我整理了一些代表性项目，从 Meta、阿里巴巴到 Nous Research，从 Karpathy 个人项目到斯坦福博士论文等方案，至少有十几个团队在不同的技术方向上探路。

梳理这些项目的技术路线，可以归为六种机制。

每一种都在回答同一个问题：怎么让 Agent 不重新训练，就能越来越强。



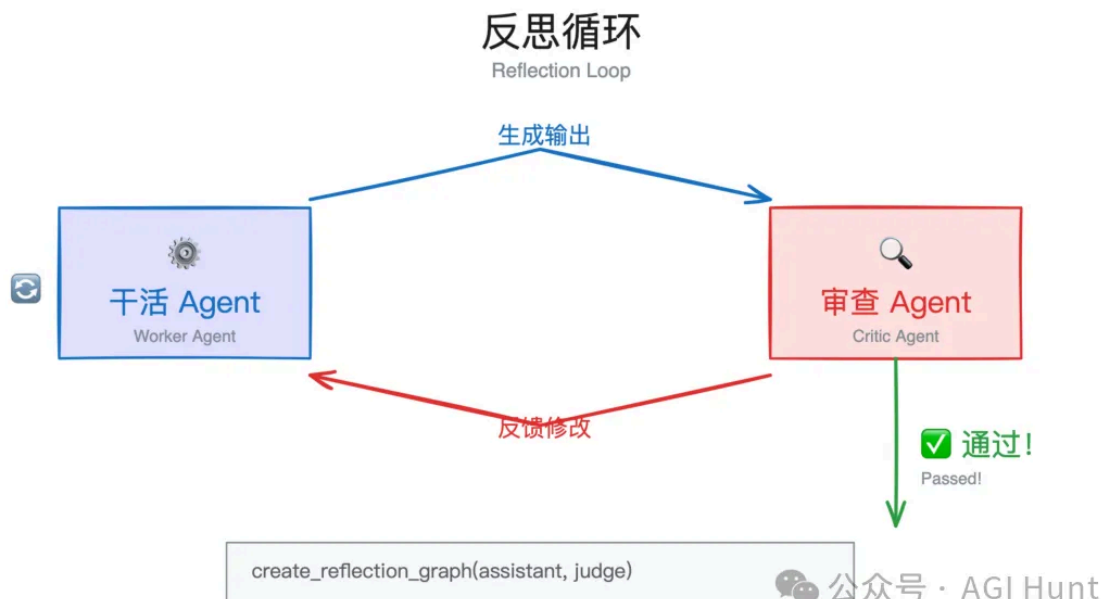
自我改进光谱：从反思到自我修改的六个层级

01

输出自审

最基础的一种：Agent 生成回答后不直接输出，先让另一个 Agent 审一遍。

技术上叫 **Reflection**，核心结构是一个双 Agent 循环。



反思循环：干活 Agent 与审查 Agent 的迭代循环

Generator 接收用户输入，生成初始回答。Critic 接收这个回答，判断有没有问题。有问题就把修改建议传回 Generator，Generator 据此重新生成。

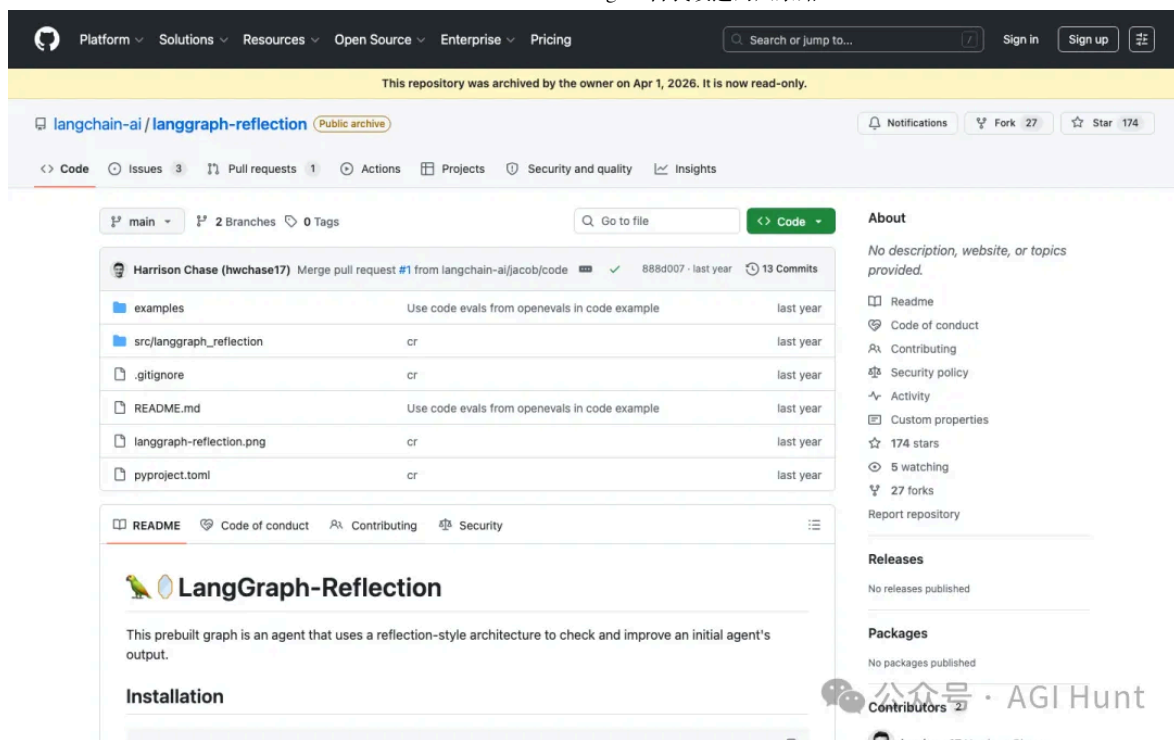
循环一直转，直到 Critic 没什么可改的了，返回空消息，循环终止。

终止条件的设计算是颇为优雅：**Critic 不返回消息 = 通过**。不需要额外评分阈值。

LangChain 的 **LangGraph Reflection** 是这个模式的标准实现，核心就一行 API：

```
create_reflection_graph(assistant_graph, judge_graph)
```

L



LangGraph Reflection GitHub 页面

在代码生成场景中，Critic 可以直接跑 Pylint 做静态类型检查，把错误作为 feedback 传回去。每一轮循环都有明确的、可验证的改进标准。

不过 Reflection 有一个硬限制：**改进只发生在单次执行内。**

下次对话开始，Agent 并不会记得上次犯过什么错。它能在当下把事情做好，但没有跨 session 的学习能力。

（这个项目今年 4 月已经归档了，模式好像已被合并进 LangGraph 核心。但 Reflection 作为一种基础模式，几乎被后面所有自我改进项目采用。）

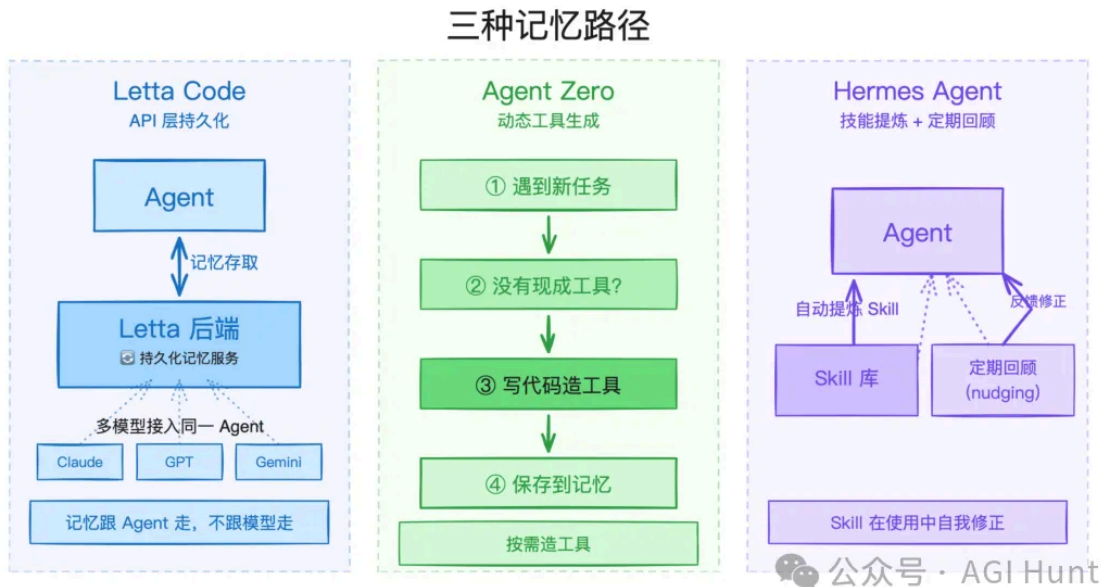
02

持久记忆

Reflection 解决了「当次做好」的问题。但「越用越好」需要另一种能力：**跨 session 的记忆持久化。**

核心思路是把 Agent 的状态从「对话级」提升到「Agent 级」。对话可以结束，但 Agent 的知识不清零。

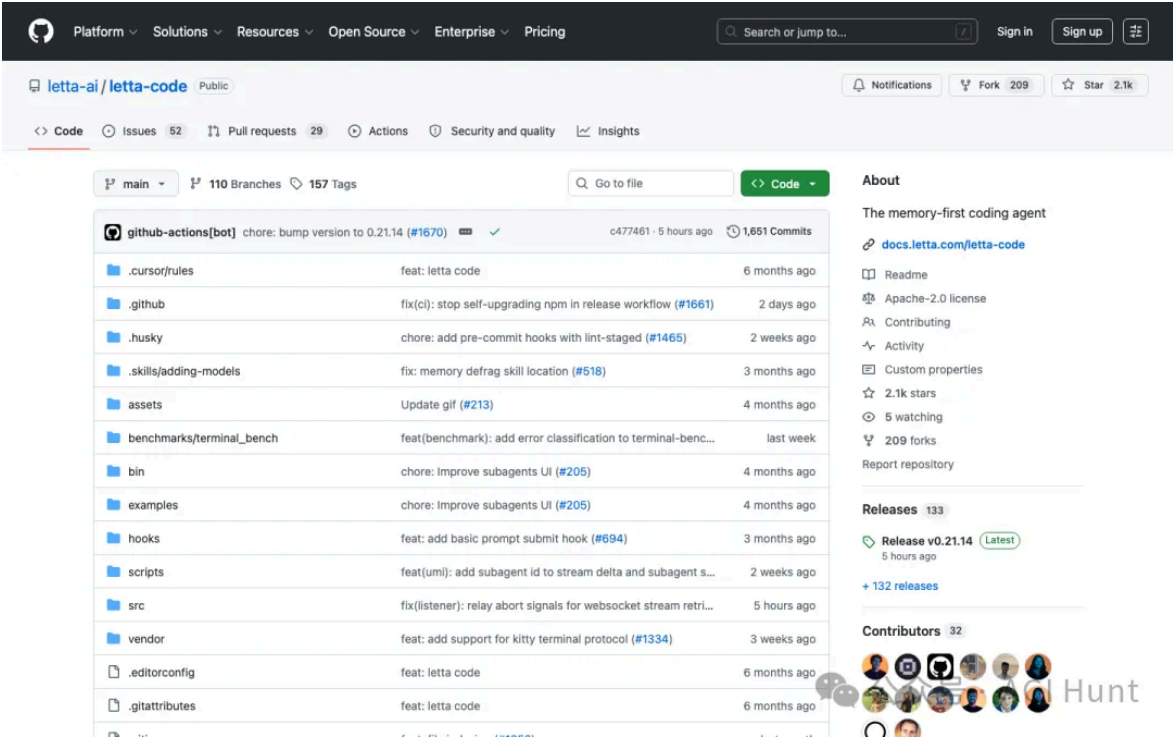
技术上有几种不同的实现路径。



三种记忆路径：API 持久化、动态工具生成、技能提炼

Letta Code 采用的是 API 层持久化。

Agent 的记忆存在 Letta 后端服务中（云端或自部署 Docker），每次新对话自动加载之前积累的状态。 `/remember` 显式写入记忆， `/skill` 把当前操作轨迹抽象成可复用的技能模块。



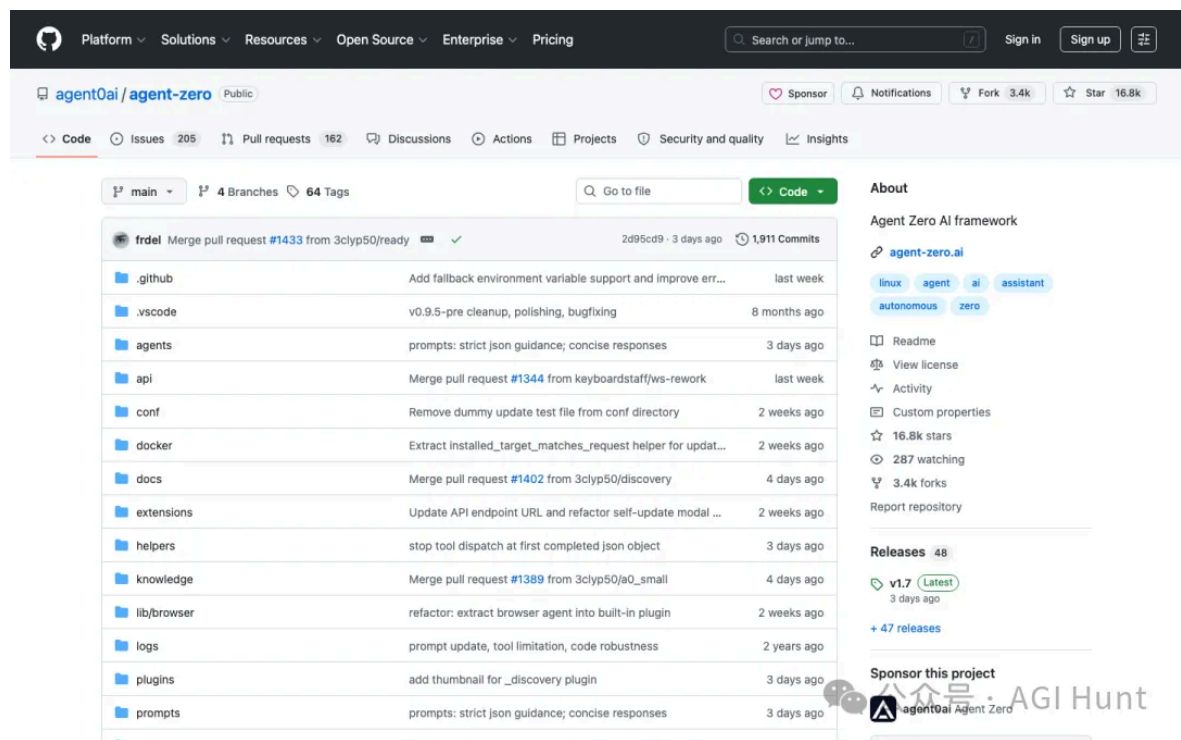
Letta Code GitHub 页面

这里有个关键的架构决策：记忆绑定在 Agent 上，不绑定在 LLM 上。

今天用 Claude，明天换 GPT，后天换 Gemini，记忆都在。同一团队做的 **LettaBot** 把这个思路延伸到多平台，一个 Agent 同时接 Telegram、Slack、Discord，所有渠道共享记忆。

Agent Zero (16,700 星) 走了另一条路：**动态工具生成 + 记忆**。

Agent 遇到没有现成工具的任务时，当场写代码创建新工具，然后存入记忆，下次直接复用。行为几乎全定义在可编辑的 prompt 文件中，没什么硬编码，小模型也能驱动。

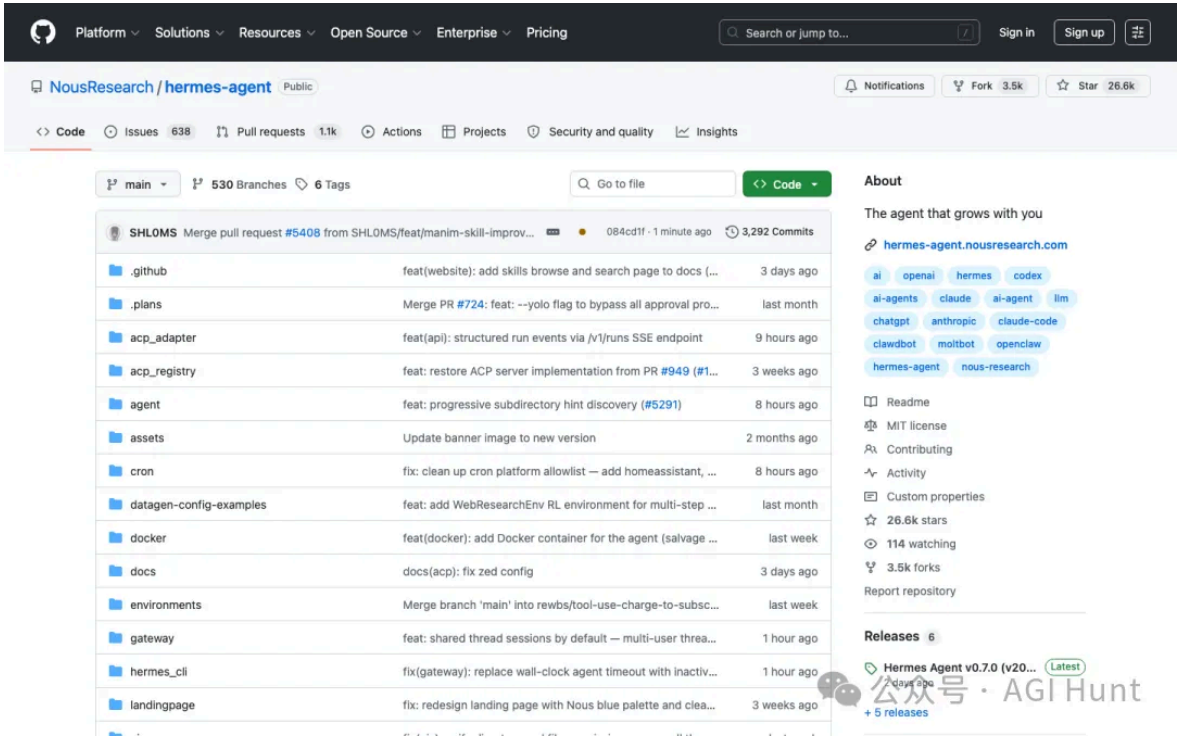


Agent Zero GitHub 页面

Hermes Agent (25,700 星) 则是目前机制最完整的。它在记忆之上加了两层：

一是**自动技能提炼**。完成复杂任务后，Agent 自动把操作步骤抽象成 Skill 文档，下次直接调用。而且 Skill 在使用过程中会持续自我修正。

二是**定期回顾** (nudging)。哪怕用户没发起对话，Agent 也会自己复盘经验，主动存下重要知识。这解决了一个实际问题：很多有用经验出现在对话中途，用户不会刻意去保存它们，但 Agent 会。



Hermes Agent GitHub 页面

这些方案的共同技术洞见是：**不改权重，改状态。**

在 LLM 参数冻结的情况下，通过外部的持久化状态层来积累知识。

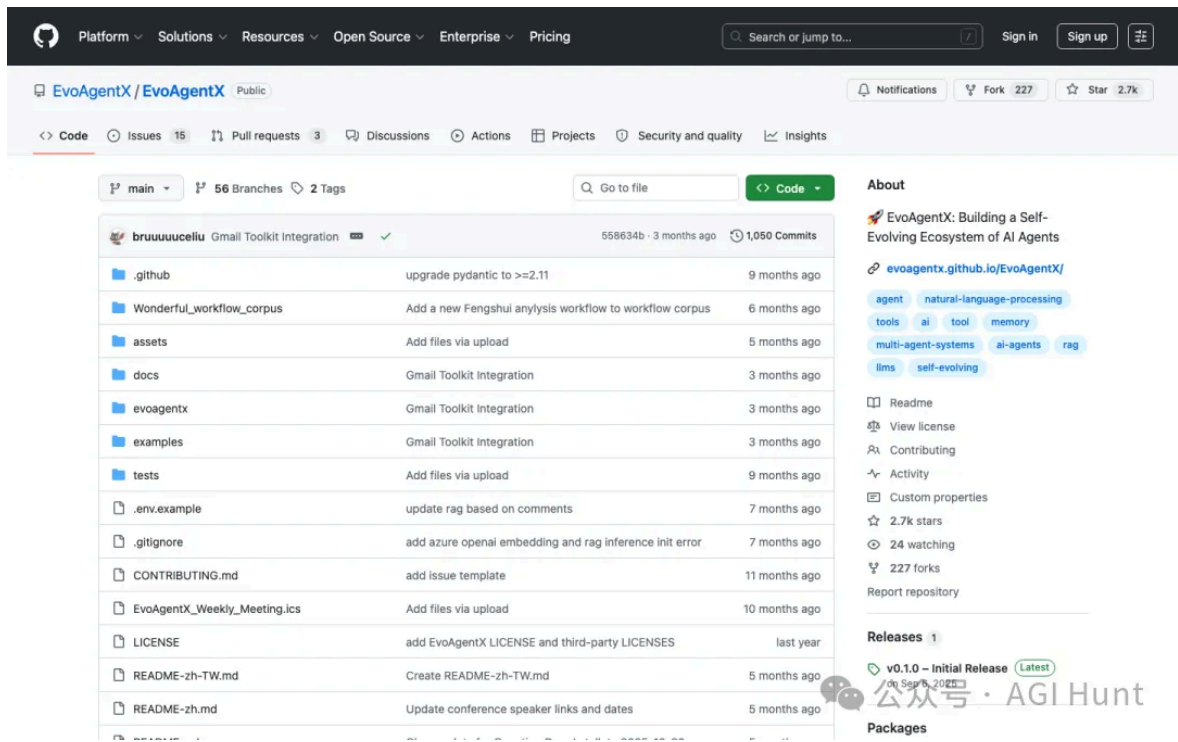
03

进化搜索

记忆解决了「记住经验」。但如果 Agent 的 prompt 写法、工具配置、 workflow 拓扑本身就有优化空间呢？

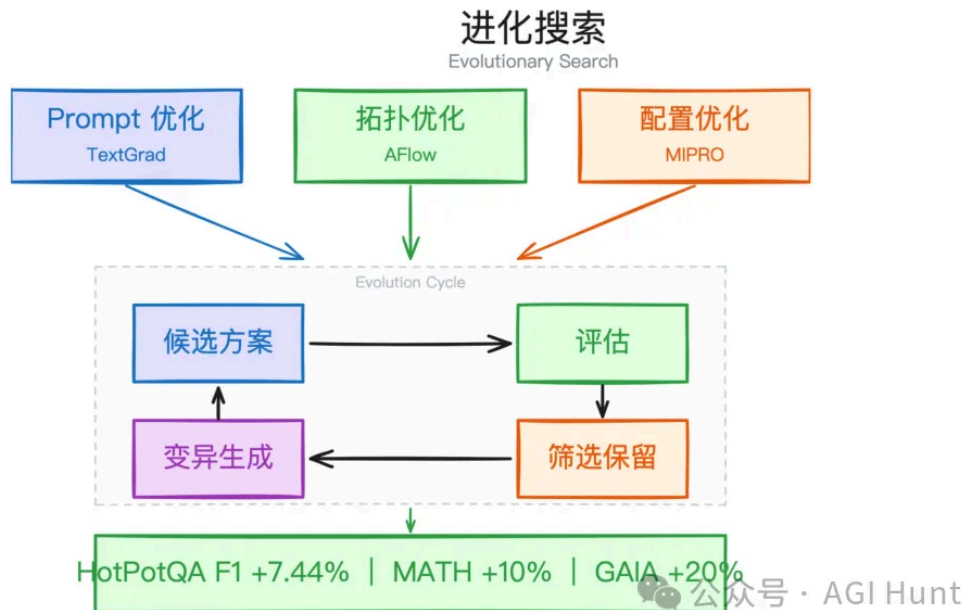
这时候需要更系统的方法：**用算法搜索更好的 Agent 配置。**

EvoAgentX（2,700 星）的做法是：给一个自然语言目标，它先自动生成多 Agent 工作流，然后用进化算法迭代优化。



EvoAgentX GitHub 页面

它同时优化三个层面：

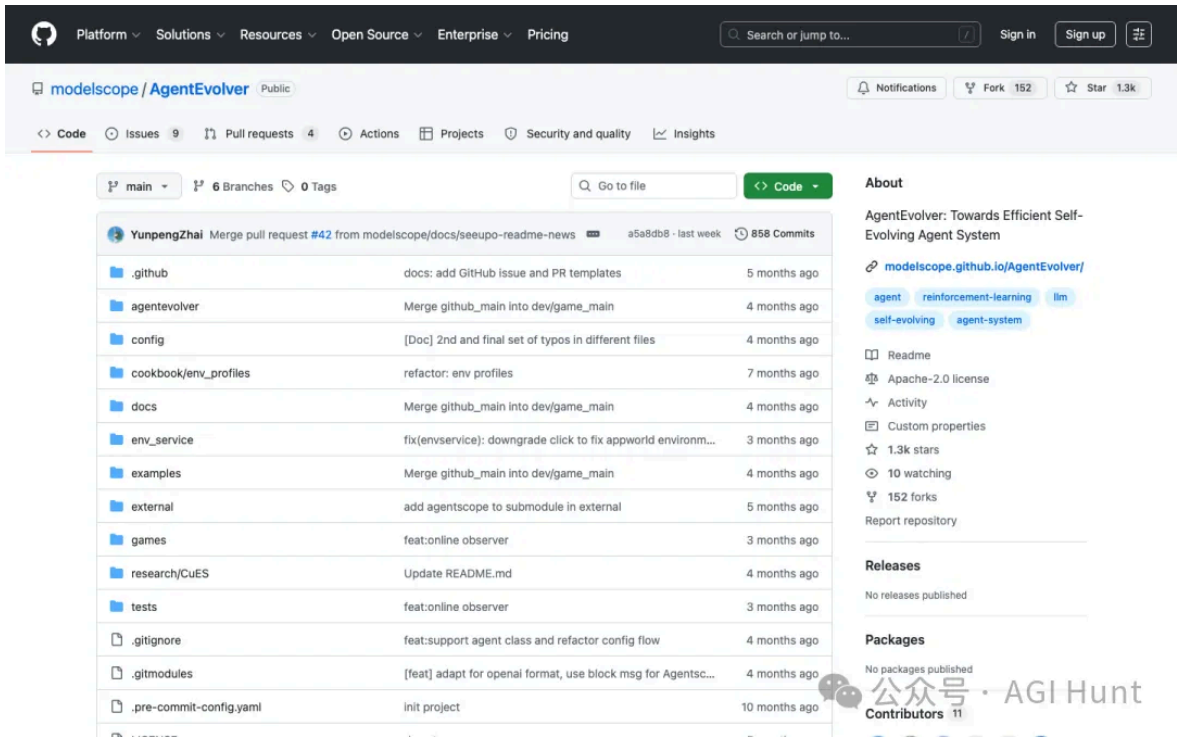


进化搜索：三条优化线并行进化

Prompt 文本，用 TextGrad 调整每个 Agent 的指令措辞。 **workflow 拓扑**，用 AFlow 搜索 Agent 间的连接方式。**配置参数**，用 MIPRO 优化工具选择和参数设定。

三条线并行进化，每一代的最优个体保留，变异出新方案，再评估、再筛选。实测 HotPotQA F1 提升 7.44%，MATH 准确率提升 10%，GAIA 综合最高提升 20%。

阿里巴巴的 AgentEvolver (1,300 星) 把进化粒度做得更细，分三个阶段：



AgentEvolver GitHub 页面

自我提问 (Self-Questioning)。 Agent 自主探索环境，给自己生成训练任务，不需要人工准备数据。

自我导航 (Self-Navigating)。 用 ReMe 经验池管理模块，把跨任务的成功经验存储起来，后续任务直接调用。

自我归因 (Self-Attributing)。 用 ADCA-GRPO 算法做轨迹级别的因果信用分配。传统 RL 给整条轨迹一个 reward，但 ADCA-GRPO 会分析每一步操作的因果贡献：第 3 步帮了多少忙，第 7 步拖了多少后腿。

这种细粒度信用分配让优化效率大幅提升。7B 模型在 AppWorld 上从 1.8% 跳到 32.4%，14B 达到 48.7%。

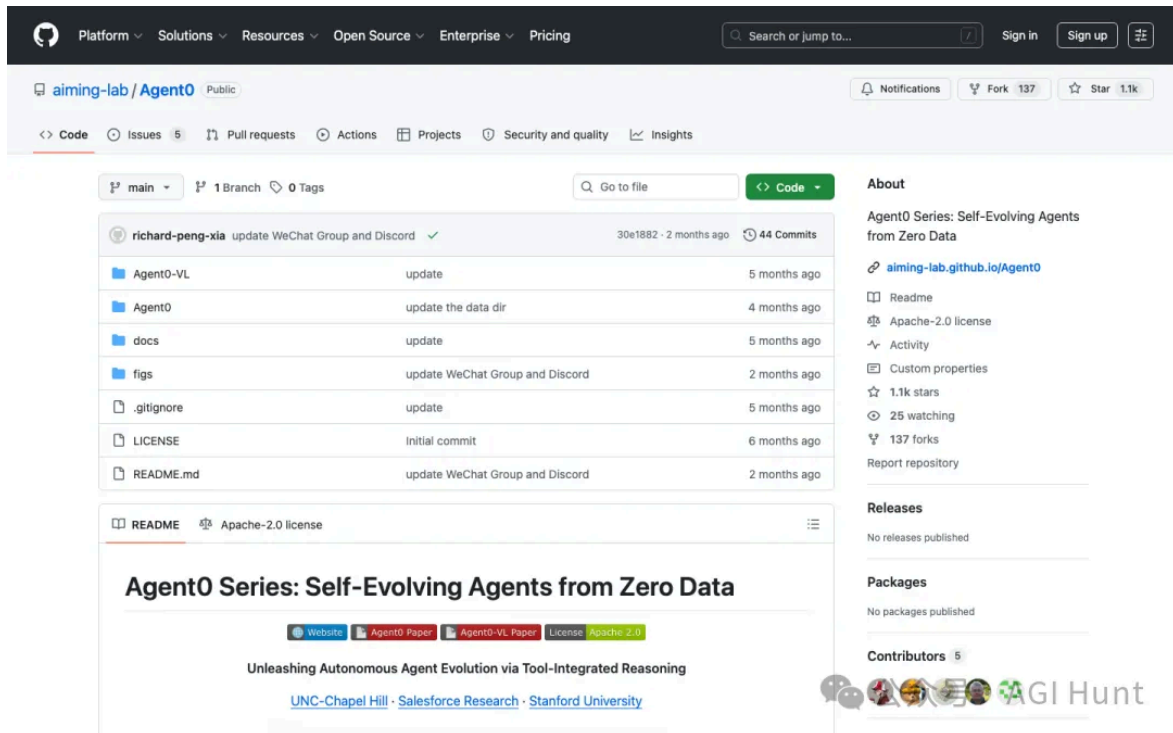
一个 7B 小模型经过自我进化，就能在特定任务上逼近数倍大的模型。

这个结论本身，就说明了进化搜索的价值。

对抗训练

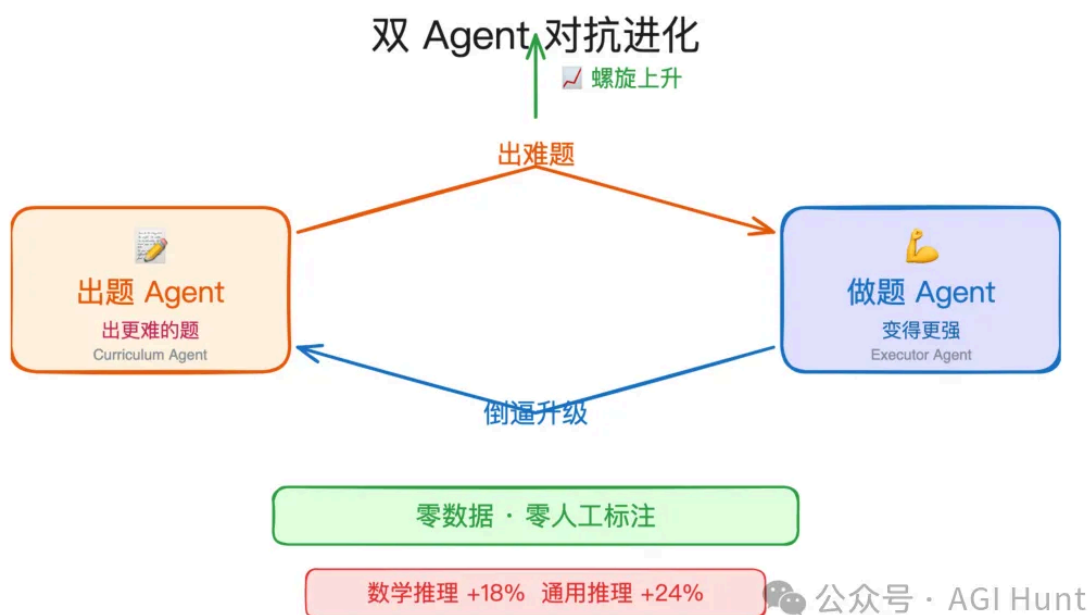
进化搜索需要评估环境来打分。但如果.....连训练数据和评估基准都没有呢？

Agent0 (1,100 星，来自北卡罗来纳大学和 Salesforce) 的方案叫**零数据自我进化**，核心机制是双 Agent 对抗。



Agent0 GitHub 页面

两个 Agent 从同一个基础模型初始化，分配到两个对立角色：



Agent0 双 Agent 对抗进化：出题与做题的螺旋上升

Curriculum Agent 生成越来越难的任务。**Executor Agent** 用工具集成推理来解题。

关键动力学在于：Executor 变强后，简单题目没有训练价值了，Curriculum Agent 就被迫生成更难的任务。更难的任务又倒逼 Executor 进化出更强能力。

竞争本身就是训练信号。 整个过程不需要人工标注数据集，也不需要外部 reward model。

效果出乎意料。

基于 Qwen3-8B-Base 的数学推理提升了 18%（达到 58.2 分），超过了需要人工标注的 R-Zero 和 Socratic-Zero。通用推理提升 24%。视觉版本 Agent0-VL 在开源视觉语言模型中排到了第一。

零标注，胜过有标注。

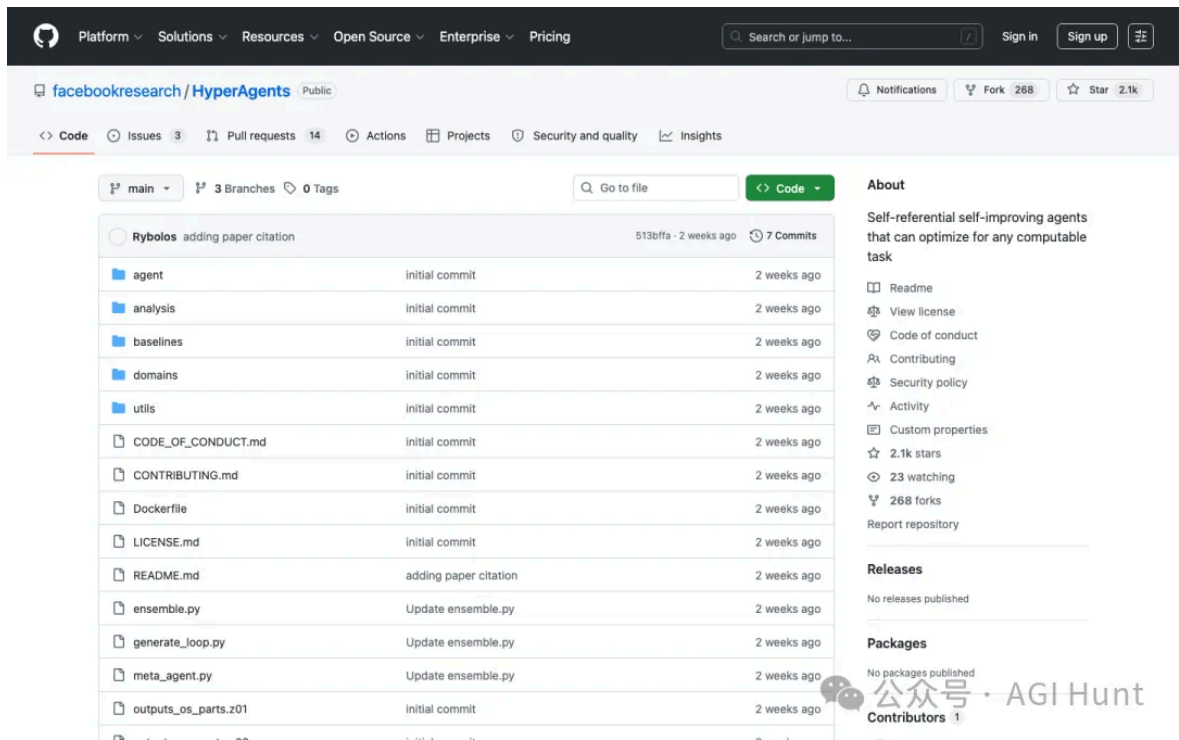
这个结论背后的含义，值得我们琢磨：也许精心策划的「对抗压力」比精心标注的数据集更能激发模型潜力。

05

自我修改

前面四种方法有一个共同前提：改进机制本身是人设计的、固定的。Reflection 的循环逻辑是人写的，进化算法是人定义的，记忆结构是人搭的。

Meta 的 **HyperAgents** (2,100 星) 打破了这个限制。

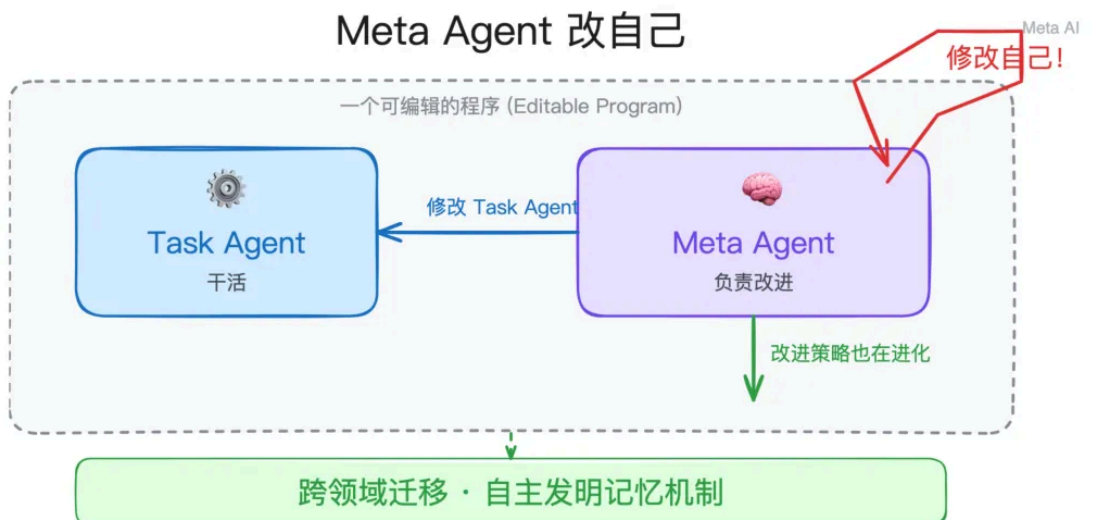


HyperAgents GitHub 页面

之前我们专门写过一篇介绍 HyperAgents 的文章，见：[Meta 放出大招：让 AI 实现自我进化，Karpathy 的 autoresearch 沦为小弟](#)

它的核心思路：**让负责改进的 Agent，也能被改进。**

论文管这个叫 DGM-Hyperagents（Darwin Gödel Machine 的扩展版）。用我们之前文章里的类比来说：以前的 AI 像一个厨师，可以按手册不断改进菜品。但手册本身是固定的。HyperAgents 让厨师学会了改手册，而且改手册的方法本身也在不断进化。



公众号 · AGI Hunt

HyperAgents 架构：Meta Agent 能修改 Task Agent，也能修改自己

系统由 Task Agent（干活）和 Meta Agent（改进）两部分组成，统一写在一个可编辑程序中。Meta Agent 不仅能改 Task Agent 的代码，还能改自己的代码。

改进的策略本身也在进化。改进改进方法的方法.....也在进化。

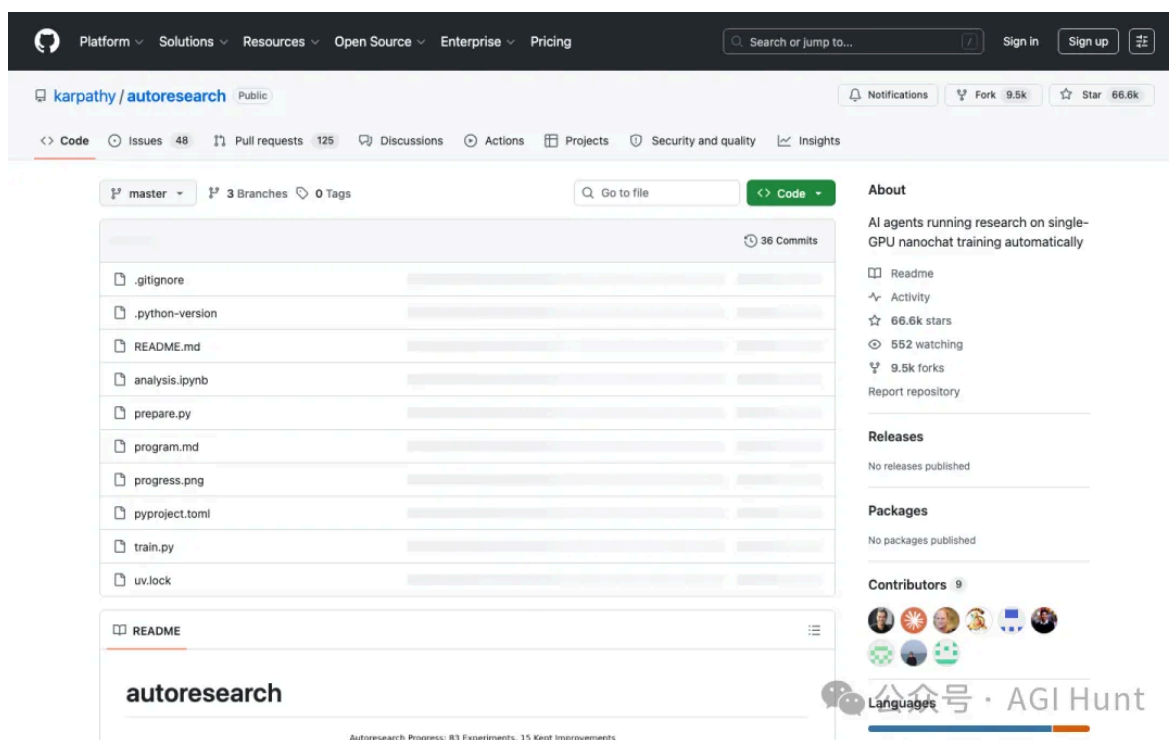
实验中最让人印象深刻的现象是：**系统自己发明了持久化记忆和性能追踪机制**。没有人预设这些功能，Agent 自己判断需要它们，就写代码给自己加上了。它记录的内容相当具体：

“ Gen55 的准确率最高但太严格了，Gen64 改善了平衡性。需要结合 Gen55 的批判性推理和 Gen64 的平衡感。

跨领域迁移的结果更值得关注。把在论文审稿和机器人任务上进化出的 HyperAgent，直接拿去做 IMO 数学评分（从没见过数学题），原版 DGM 的迁移效果约等于 0，而 DGM-H 的 imp@50 达到了 **0.630**。

原因在于：DGM 学到的是领域特定技巧，换了领域就失效。DGM-H 学到的是通用的「如何改进」策略，比如持久化记忆、趋势分析，这些在新领域照样管用。

说到这里，值得提一下 Karpathy 的 autoresearch（66,500 星），它让 AI 在夜间自动跑 ML 训练实验。



autoresearch GitHub 页面

autoresearch 和 HyperAgents 的区别恰好说明了「自动化」和「自我改进」的边界：autoresearch 中的 AI（比如 Claude Code）本身是不变的，它能得到更好的实验结果，但不会改进「自己做实验的方法」。HyperAgents 进化的不只是结果，还有进化过程本身。

06

编排自优化

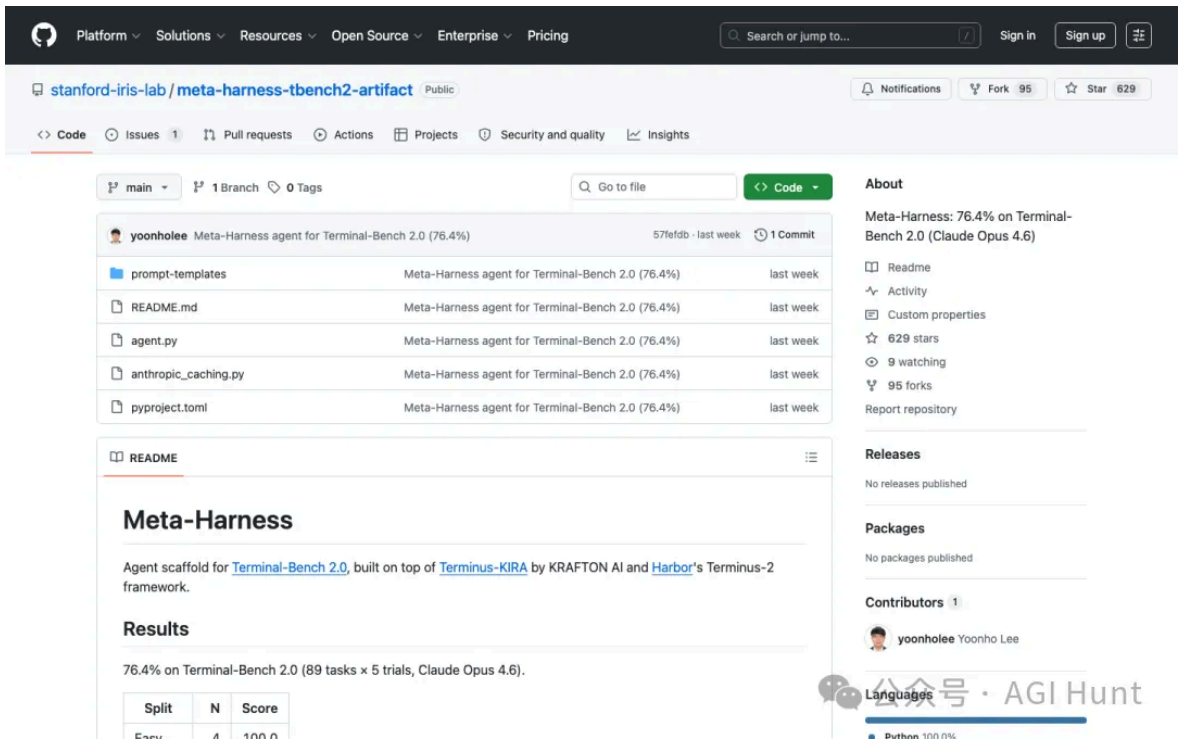
前五种方法都在改 Agent 自身。但 Agent 的表现还取决于另一个东西：围绕它的编排层（Harness）。

Harness 指的是模型之外的那一层编排逻辑：prompt 结构、检索策略、工具调用顺序、状态管理。同一个模型换一套 Harness，性能可能翻倍。

那.....Harness 的设计本身能不能也自动化？

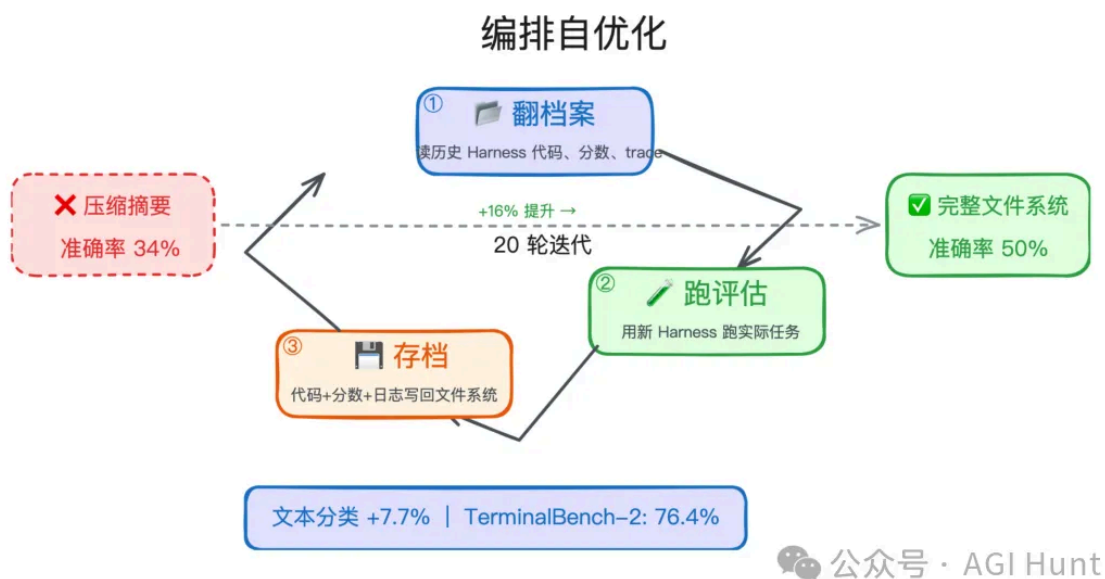
斯坦福的 **Meta-Harness**（629 星）回答了这个问题。论文一作 Yoonho Lee 是切尔西·芬恩的博士生，另一位作者是 DSPy 的作者 Omar Khattab。

我其实之前就有写过：[📄 Meta-Harness：斯坦福让 harness 实现自我改进](#)



Meta-Harness GitHub 页面

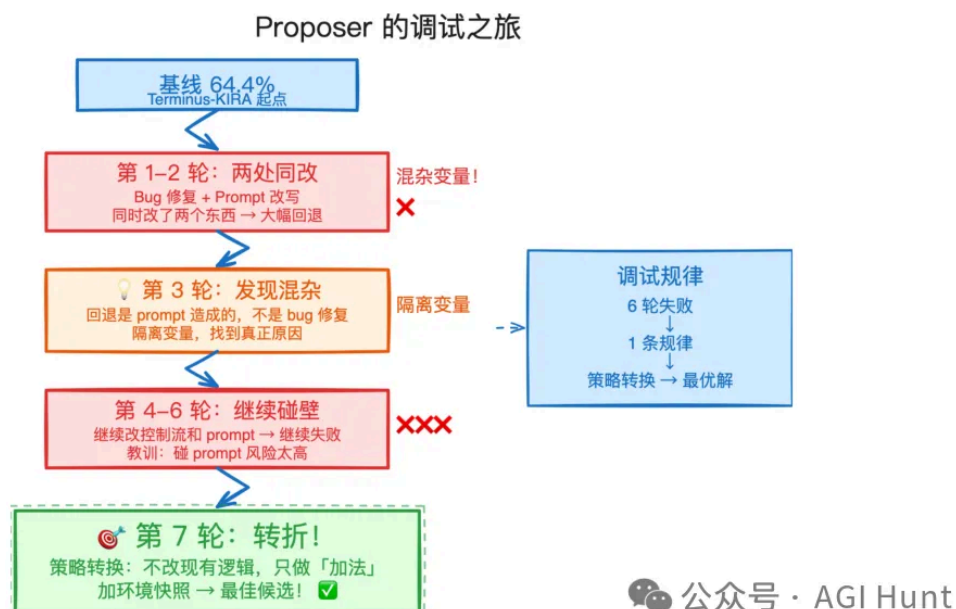
方案本身倒是挺为简洁：用一个 Coding Agent（Claude Code + Opus 4.6）来迭代优化 Harness。每一轮，Agent 读取文件系统中所有的历史记录（之前每一版 Harness 的源代码、评估分数、执行 trace），提出新的 Harness 方案，跑评估，把结果写回文件系统。



编排自优化：翻档案→跑评估→存档的迭代循环

这个过程和人类工程师调试代码的思路几乎一样。论文附录里有一段让人拍案叫绝的调试轨迹：

Agent 第 1-2 轮同时修了一个 bug 和改了 prompt 模板，性能大幅回退。第 3 轮，它回去对比两个失败候选的改动记录，发现共同点是都改了 prompt 模板，于是做了一个经典的**混杂变量识别**：把结构修复和 prompt 变更拆开，分别测试。



Proposer 的调试之旅：从失败到转折

第 7 轮，它换了策略，不再改任何现有逻辑，只在第一次 LLM 调用前加一个环境快照。这个候选成了全局最优。

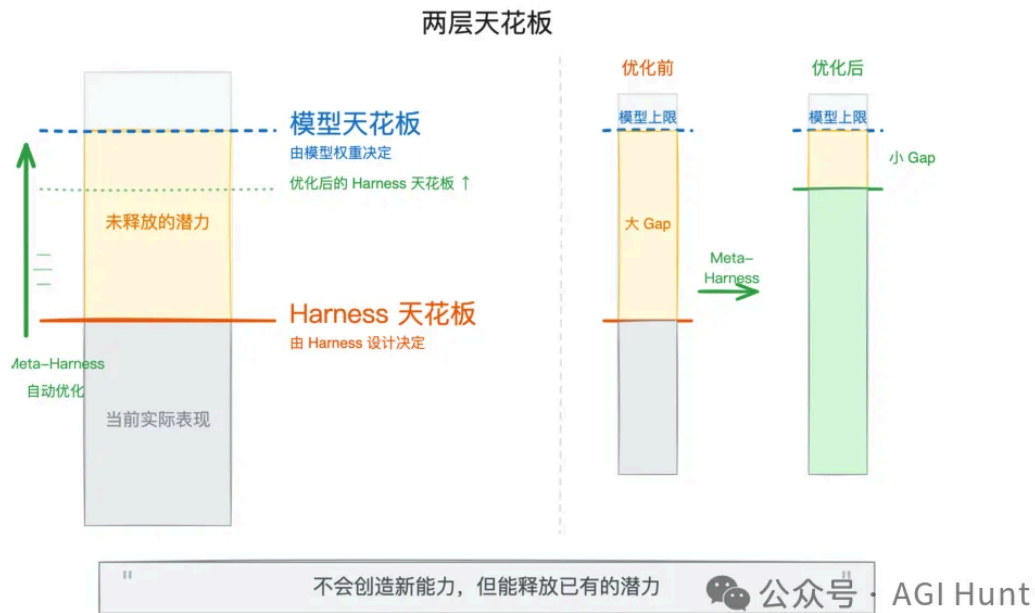
整个过程就是一个工程师的调试思路：尝试、失败、识别混杂因素、隔离变量、切换策略、组合已验证的修复。

设计上有一个关键选择：给 Agent 完整的文件系统访问权限，取代压缩摘要。

消融实验显示，只给分数和摘要时中位数准确率 34%，给完整文件系统直接跳到 50%。摘要版最高准确率（38.7%）甚至不如完整版的中位数（50.0%）。

压缩不只是丢了边角细节，而是丢掉了做正确决策所需的关键线索。

结果在文本分类上比人工最优方案 ACE 高 7.7 个百分点，context 用量只有 ACE 的四分之一。在 TerminalBench-2 上拿到 76.4% 通过率，超过人工精调的方案，在所有 Opus 4.6 Agent 中排名第二。



两层天花板：模型上限与 Harness 上限

Big Model 和 Big Harness，两层天花板，缺一不可。

模型能力决定了理论上限，Harness 决定了实际达到的高度。Meta-Harness 做的事情，是把 Harness 这一层的天花板尽量往模型天花板靠近。

而 Meta-Harness 和前面五种方法的区别在于：它改的是 Agent 周围的「脚手架」，并非 Agent 自身。

但效果是一样的，系统整体在自动变强。

07

回到同一个问题

六条路，每条都从不同角度回答了同一个问题：怎么让 Agent 在不重新训练的情况下变强。

机制	核心思路	代表项目	Stars
输出自审	生成后审查，循环修正	LangGraph Reflection	173
持久记忆	跨 session 积累知识和技能	Letta Code · Agent Zero · Hermes Agent	2.1k · 16.7k · 25.7k

机制	核心思路	代表项目	Stars
进 化 搜 索	用算法优化 prompt、工具和工作流	EvoAgentX · AgentEvolver	2.7k · 1.3k
对 抗 训 练	双 Agent 竞争产生训练信号	Agent0	1.1k
自 我 修 改	改写自己的代码和改进机制	HyperAgents	2.1k
编 排 自 优化	自动优化 Agent 的编排层	Meta–Harness	629

六种机制并非互斥。

Hermes Agent 同时用了反思、记忆和技能进化。AgentEvolver 同时做了自我提问（对抗生成的变体）和进化搜索。Meta–Harness 的内部循环本身也包含反思和进化。

它们共同指向的核心命题：**AI 的学习，正在从训练阶段溢出到部署阶段。**

过去十年，模型变强的唯一方式是改权重。这些项目展示了另一种可能：权重冻结的情况下，通过外部记忆、行为搜索、对抗训练、代码自修改、编排自优化来持续积累能力。

如果说，训练是「上学」

那这些机制，就是毕业之后的.....

自学能力。



相关链接：

- HyperAgents (Meta): <https://github.com/facebookresearch/Hyperagents>
- Agent0: <https://github.com/aiming-lab/Agent0>

- EvoAgentX: <https://github.com/EvoAgentX/EvoAgentX>
- AgentEvolver: <https://github.com/modelscope/AgentEvolver>
- Agent Zero: <https://github.com/frdel/agent-zero>
- Letta Code: <https://github.com/letta-ai/letta-code>
- Hermes Agent: <https://github.com/NousResearch/hermes-agent>
- autoresearch (Karpathy): <https://github.com/karpathy/autoresearch>
- Meta-Harness (Stanford): <https://github.com/stanford-iris-lab/meta-harness-tbench2-artifact>
- LangGraph Reflection: <https://github.com/langchain-ai/langgraph-reflection>

